Converting actor `Cargos` to .proto `Cargos` is intuitive, but it may not be obvious how it returns a `Future` after the `map`. If you look at the signature of `mapTo`, it becomes clear (probably not how it works, but at least that a `Future` is produced):

```
def mapTo[S](...): Future[S]
```

With the latest implementation of the service, you can bind it and create the application just as before (shown again for completeness in the following listing).

Listing 11.10 Application with an actor and gRPC port

```
object ContainerServerSharding {

  def main(args: Array[String]): Unit = {

    implicit val system =
      ActorSystem(Behaviors.empty, "ContainerServer")

    implicit val ec: ExecutionContext = system.executionContext
    val sharding: ClusterSharding = ClusterSharding(system)

    val shardingRegion =
      sharding.init(Entity(Container.TypeKey)(entityContext =>
        Container(entityContext.entityId)))

    val service: HttpRequest => Future[HttpResponse] =      ❶
      ContainerServiceHandler.withServerReflection(         ❷
        new ContainerServiceImplSharding(sharding))         ❸

    val bindingFuture: Future[Http.ServerBinding] =
      Http().newServerAt("localhost", 8080).bind(service)   ❹

    println(s"server at localhost:8080 \nPress RETURN to stop")
    StdIn.readLine()                                        ❺
    bindingFuture    ()                                     ❺
      .flatMap(_.unbind())()                                ❺
      .onComplete(_ => system.terminate())()                ❺
```